

MiniBench ??????

agents / core / factory ?????? OpenAI-compatible?direct ? prompt ????

??

???	D:\AAALimoWork\CS\Seminar\MiniBench
???	src/minibench/agents, src/minibench/core, src/minibench/factory
???	????? CLI ??? factory ??? agent ? task family?dataset evaluation ?? Agent.generate?core ?? prompt?metrics?summary ??????

????MiniBench ???

????????MiniBench ??????????????????????

- agents???????? agent ??? generate(prompt, task) -> str?
- core???????????? prompt?metrics?summary?run ?????
- factory????? YAML/CLI ?????? agent?provider?task family ? evaluation?

YAML/config

- > factory.config.validate_experiment_config
- > factory.experiments.get_task_family_spec
- > load_tasks
- > factory.agents.make_agent_from_config
- > dataset.evaluate_tasks(..., agent)
- > core.metrics / core.results / core.runs
- > runs/<run_name>/predictions.jsonl, results.json, summary.txt

???MiniBench ??? agent ??? task family ????????????????

??agents ???

agents ????????????????????????????????????? Agent.generate ?????? prompt ? task????????????????????

__init__.py

???????????????? Agent?ChatClient?ReasoningConfig ?????? agent ??????????????????????
minibench.agents ???

simple.py

???????? agent????? baseline?????????

- OracleAgent????? task.correct_option????? JSON????????????????????
- NoisyAgent????????????????? answer: A???????????????????? JSON?
- PredictionFileAgent?? JSONL ?????? raw_output/raw_outputs?? task.id ????????????????

direct.py

DirectAgent ????????????????????????????????????? direct_prompt ???? prompt????????? JSON?

prompt -> direct_prompt(prompt)
-> client.complete(..., FINAL_ANSWER_SYSTEM_PROMPT, json_mode=True)
-> JSON answer

???????? prompt-engineering baseline?

cot.py

CoTAgent ? Chain-of-Thought???????????????????????????????????? JSON?

reasoning = client.complete(cot_prompt(prompt), json_mode=False)
answer = client.complete(finalize_prompt(prompt, reasoning), json_mode=True)

??

critic_refine.py

CriticRefineAgent ?????????????? self-refinement ????

- draft?? direct_prompt ??????
- critique?? critic_prompt ?????????? schema?
- refine?? refine_prompt ?????????? JSON?

plan_then_solve.py

PlanThenSolveAgent ?????????? CoT ?????CoT ?????Plan-Then-Solve ????? plan??? plan ???

```
plan      = client.complete(plan_prompt(prompt))
solution = client.complete(solve_with_plan_prompt(prompt, plan))
answer    = client.complete(finalize_prompt(prompt, solution), json_mode=True)
```

self_consistency.py

SelfConsistencyAgent ?????????????? samples ? CoT ?????????????? A/B/C/D?

- ?????????????? JSON?
- ?????????????? judge_prompt ??????????????

???????????????????? reasoning_temperature ????

tree_of_thought.py

TreeOfThoughtAgent ??? Tree of Thought???????????????? judge_prompt ???????

```
candidates = [candidate_prompt(prompt, i) for i in range(samples)]
answer = client.complete(judge_prompt(prompt, candidates), json_mode=True)
```

Self-consistency ?????Tree-of-Thought ??????????

```
agents ?????agents ????????????????? generate ????????????? direct?CoT?ToT ??
critic-refine???? generate ???
```

??core ???

core ?????????????????????????????? benchmark ??????????

__init__.py

???? Agent?ChatClient?ReasoningConfig??? core ??? API ???

agent.py

????? agent ??????

- Agent???????????? generate(prompt, task) -> str?
- ChatClient?Protocol ???????? complete(...) ??????????????????
- ReasoningConfig?????
samples?reasoning_temperature?final_temperature?max_reasoning_tokens?final_max_tokens?

????????????agent ????? ChatClient.complete?????? provider ? DeepSeek?Qwen?SiliconFlow ??
generic API?

prompts.py

?????? prompt ????????? system prompt?

- FINAL_ANSWER_SYSTEM_PROMPT????????? JSON object?
- REASONING_SYSTEM_PROMPT????????????????????
- CRITIC_SYSTEM_PROMPT????????????????? schema?

?? prompt builder ?? direct_prompt?cot_prompt?plan_prompt?solve_with_plan_prompt?candidate_prompt?judge_prompt?finalize_prompt?critic_prompt?refine_prompt?

??? prompt ????????? benchmark adapter????????? prompt ??? agent ??????????????????????

metrics.py

metrics.py ?????????????????LLM ????? token usage????????? snapshot-delta?

```
start = start_task_metrics(agent)
raw_output = agent.generate(prompt, task)
metrics = finish_task_metrics(agent, start)
```

- start_task_metrics??? perf_counter ????????
- finish_task_metrics????????? after - before ???????
- model_metrics_snapshot????? agent ? agent.client ?? metrics_snapshot?
- summarize_metrics????????? total ? average_per_task?
- extract_token_usage????? provider ??? usage???
prompt_tokens?completion_tokens?total_tokens ??? *_tokens ???

????????? agent????? generate ?????? LLM ???

results.py

results.py ??? summarize_boolean_results???????????????? benchmark?

- ??? total?success_count ? rate?

- ? tags ???? by_tag?
- success_attr?success_key?rate_key ?????????????? correct/accuracy ? success/success_rate
?????????

runs.py

runs.py ??????????????

- timestamped_run_dir??? run_name?prefix ?????? run ???exist_ok=False ????????
- write_jsonl?? dataclass ? dict ?? JSONL???? ensure_ascii=False ?????
- write_summary_artifacts???? predictions.jsonl?results.json?summary.txt?

core ?????core ??? MiniBench ?????????? agent ???prompt ???metrics ??????????????????

- run.output_dir ????? runs?

????????????????????????????

experiments.py

experiments.py ? factory ???????? TaskFamilySpec ????????????????

TaskFamilySpec:

default_path
load_tasks
evaluate_tasks
summarize
write_run
system_prompt

?? TASK_FAMILIES ?? multiple_choice?xiangqi?one_stroke?mahjong?mahjong_riichi??? family
?????? load/evaluate/summarize/write_run ???

- run_family_experiment???????? task_ids??? limit??? agent??? evaluation??? run?

- _select_tasks?? id ??????????? id ???

- _evaluate?? xiangqi ? mahjong_riichi ??????????? family ?? evaluate_tasks(tasks, agent)?

factory ?????factory ??????? task?agent?provider ? run ???????????????? task family ??????
load/evaluate/summarize/write_run ??? TASK_FAMILIES??????????????

OpenAI-compatible ? direct

openai-compatible ? direct agent

openai-compatible	provider client dataset prompt provider/default system prompt
direct	direct_prompt prompt JSON?

openai-compatible:

prompt -> OpenAICompatibleAgent.complete(prompt) -> raw output

direct:

```
prompt -> direct_prompt(prompt)
        -> OpenAICompatibleAgent.complete(
            system_prompt=FINAL_ANSWER_SYSTEM_PROMPT,
            json_mode=True
        )
        -> JSON answer
```

???

??	openai-compatible	direct
??	provider/base model agent	?? prompt-engineering ??
???? prompt	????	?? direct_prompt ??
JSON mode	??? provider ??	???? json_mode=True
????? agent ??? client	?	????
????	1 ?	1 ?

openai-compatible prompt direct
agent baseline openai-compatible baseline?

prompt.py

prompts.py token prompt

prompt

??????

- few-shot Tree-of-Thoughts repo ? Game of 24 prompt
- ToT propose_prompt?value_prompt?vote_prompt? prompt?
- judge rubric? prompt correctness?constraint following?schema compliance?confidence?
- crossword?game24 confidence
- Auto-CoT demonstration? few-shot CoT
- Self-Refine?initial output -> feedback -> refinement?critic prompt ? refine prompt

??????

- direct_prompt???????????????? JSON ???
- cot_prompt???????? Analysis ? Answer????? finalize_prompt ?? JSON?
- candidate_prompt???? candidate ??????????
- judge_prompt???? rubric???????? JSON?
- critic_prompt?????????????schema????????
- task family prompt???????????????? reasoning prompt?

prompt ?????? prompt ?????????? prompt ?????????? prompt engineering?????????
JSON schema ??????????????????????????

??????

??????????

MiniBench ??????????????????agent ??????core ??????????????factory
?????????OpenAI-compatible ??????direct/CoT/ToT/self-consistency ?????????? prompt
????????? benchmark ?????????????????? ToT?Auto-CoT?Self-Refine ??????
few-shot?rubric????????? prompt?

????

- DeepSeek API Docs: <https://api-docs.deepseek.com/>
- Tree of Thoughts official repo prompts:
<https://github.com/princeton-nlp/tree-of-thought-llm>
- Auto-CoT repo: <https://github.com/amazon-science/auto-cot>
- Self-Consistency paper: <https://arxiv.org/abs/2203.11171>
- Self-Refine paper: <https://arxiv.org/abs/2303.17651>